

Nombre de la práctica	Regresión Logística			No.	5
Asignatura:	Simulación	Carrera:	ISIC	Duración de la práctica (Hrs)	2

I. Competencia(s) específica(s): Aprender Regresión Logística con python

II. Lugar de realización de la práctica (laboratorio, taller, aula u otro): Aula

III. Material empleado:

Computadora

Python

IV. Desarrollo de la práctica:

Regresion logistica: Deteccion de SPAM

En este ejercicio se muestran los fundamentos de la regresion logistica planteando uno de los primeros problemas que fueron solucionados mediante el uso de tecnicas de Machine Learning: la Deteccion de SPAM

Enunciado del ejercicio

Se propone la construccion de un sistema de aprendizaje automatico capaz de predecir si un correo determinado se corresponde con un correo SPAM o no, para ello se utilizara el siguiente DataSet:

[2007 TREC Public Spam Corpus](#)

The corpus trec07p contains 75,419 messages:

```

25220 ham
50199 spam

```

These messages constitute all the messages delivered to a particular server between these dates:

```

Sun, 8 Apr 2007 13:07:21 -0400
Fri, 6 Jul 2007 07:04:53 -0400

```



```
# En esta clase se facilita el procesamiento de correos electronicos que poseen codigo HTML
```

```
from html.parser import HTMLParser
class MLStripper(HTMLParser):
    def __init__(self):
        self.reset()
        self.strict = False
        self.convert_charrefs = True
        self.fed = []

    def handle_data(self, d):
        self.fed.append(d)

    def get_data(self):
        return ''.join(self.fed)
```

```
# Esta funcion se encarga de eliminar los tags HTML que se encuentren en el texto de los correos electronicos
```

```
def strip_tags(html):
    s = MLStripper()
    s.feed(html)
    return s.get_data()
```

```
#Ejemplo de eliminacion de los tags HTML de un texto
```

```
t = '<tr><td align="left"><a href=" ../../issues/51/16.html#article">Phrack World News </a><td>'
strip_tags(t)
```

```
'Phrack World News '
```

Ademas de eliminar los posibles tags HTML que se encuentran en el correo electronico deben reslizarse ptras opciones para evitar que los mensajes contegan ruido inecesario. Entre ellas se encuentra la eliminacion de los signos de puntuacion, eliminacion de los posibles campos de correo electronico que no sean relevantes o eliminacion de los afijos de un a palabra manteniendo unicamente la raiz de la misma (Stemming). La clase que se muestra a continuacion realiza estas transformaciones.



```
import email
import string
import nltk
class Parser:
    def __init__(self):
        self.stemmer = nltk.PorterStemmer()
        self.stopwords = set(nltk.corpus.stopwords.words('english'))
        self.punctuation = list(string.punctuation)

    def parse(self, email_path):
        """Parse an email."""
        with open(email_path, errors='ignore') as e:
            msg = email.message_from_file(e)
            return None if not msg else self.get_email_content(msg)

    def get_email_content(self, msg):
        """Extract the email content."""
        subject = self.tokenize(msg['Subject']) if msg['Subject'] else []
        body = self.get_email_body(msg.get_payload(),
                                   msg.get_content_type())
        content_type = msg.get_content_type()
        # return the content of the email
        return {"subject": subject,
                "body": body,
                "content_type": content_type}
```



```
def get_email_body(self, payload, content_type):
    """Extract the body of the email."""
    body = []
    if type(payload) is str and content_type == 'text/plain':
        return self.tokenize(payload)
    elif type(payload) is str and content_type == 'text/html':
        return self.tokenize(strip_tags(payload))
    elif type(payload) is list:
        for p in payload:
            body += self.get_email_body(p.get_payload(),
                                       p.get_content_type())
    return body

def tokenize(self, text):
    """Transform a text string in tokens. Perform two main actions,
    clean the punctuation symbols and do stemming of the text"""
    for c in self.punctuation:
        text = text.replace(c, " ")

    text = text.replace("\t", " ")
    text = text.replace("\n", " ")
    tokens = list(filter(None, text.split(" ")))
    #stemming of tokens
    return [self.stemmer.stem(w) for w in tokens if w not in self.stopwords]
```



```
inmail = open("datasets/trec07p/data/inmail.1").read()
print(inmail)
```

```
From RickyAmes@aol.com Sun Apr 8 13:07:32 2007
Return-Path: <RickyAmes@aol.com>
Received: from 129.97.78.23 ([211.202.101.74])
    by speedy.uwaterloo.ca (8.12.8/8.12.5) with SMTP id l38H7G0I003017;
    Sun, 8 Apr 2007 13:07:21 -0400
Received: from 0.144.152.6 by 211.202.101.74; Sun, 08 Apr 2007 19:04:48 +0100
Message-ID: <WYADCKPDFWWTXTXNFVUE@yahoo.com>
From: "Tomas Jacobs" <RickyAmes@aol.com>
Reply-To: "Tomas Jacobs" <RickyAmes@aol.com>
To: the00@speedy.uwaterloo.ca
Subject: Generic Cialis, branded quality@
Date: Sun, 08 Apr 2007 21:00:48 +0300
X-Mailer: Microsoft Outlook Express 6.00.2600.0000
MIME-Version: 1.0
Content-Type: multipart/alternative;
    boundary="--8896484051606557286"
X-Priority: 3
X-MSMail-Priority: Normal
Status: R0
Content-Length: 988
Lines: 24

----8896484051606557286
Content-Type: text/html;
Content-Transfer-Encoding: 7Bit
```

```
<html>
<body bgcolor="#ffffff">
<div style="border-color: #00FFFF; border-right-width: 0px; border-bottom-width: 0px; margin-bottom: 0px;" align="center">
<table style="border: 1px; border-style: solid; border-color:#000000;" cellpadding="5" cellspacing="0" bgcolor="#CCFFAA">
<tr>
<td style="border: 0px; border-bottom: 1px; border-style: solid; border-color:#000000;">
<center>
Do you feel the pressure to perform and not rising to the occasion??<br>
</center>
</td></tr><tr>
<td bgcolor=#FFFF33 style="border: 0px; border-bottom: 1px; border-style: solid; border-color:#000000;">
<center>
<b><a href='http://excoriationtuh.com/?lzmfnrdkleks'>Try <span>V</span><span>ia<span></span>gr<span>a</span>....</a></b></center>
</td></tr><td><center>your anxiety will be a thing of the past and you will<br>
be back to your old self.
</center></td></tr></table></div></body></html>
```

Parsin del correo electronico

```
p = Parser()
p.parse("datasets/trec07p/data/inmail.1")

{'subject': ['gener', 'ciali', 'brand', 'qualiti'],
 'body': ['do',
 'feel',
 'pressur',
 'perform',
 'rise',
 'occas',
 'tri',
 'viagra',
 'anxieti',
 'thing',
 'past',
 'back',
 'old',
 'self'],
 'content_type': 'multipart/alternative'}
```

Lectura del indice

Estas funciones complementarias se encargan de cargar en memoria la ruta de cada correo electronico y su etiqueta correspondiente. {spam y ham}

```
index = open("datasets/trec07p/full/index").readlines()
index
```

```
['spam ../data/inmail.1\n',
 'ham ../data/inmail.2\n',
 'spam ../data/inmail.3\n',
 'spam ../data/inmail.4\n',
 'spam ../data/inmail.5\n',
 'spam ../data/inmail.6\n',
 'spam ../data/inmail.7\n',
 'spam ../data/inmail.8\n',
 'spam ../data/inmail.9\n',
 'ham ../data/inmail.10\n',
 'spam ../data/inmail.11\n',
 'spam ../data/inmail.12\n',
 'spam ../data/inmail.13\n',
 'spam ../data/inmail.14\n',
 'spam ../data/inmail.15\n',
 'spam ../data/inmail.16\n',
 'spam ../data/inmail.17\n',
 'spam ../data/inmail.18\n']
```



```
import os

DATASET_PATH = "datasets/trec07p"

def parse_index(path_to_index, n_elements):
    ret_indexes = []
    index = open(path_to_index).readlines()
    for i in range(n_elements):
        mail = index[i].split(" ../")
        label = mail[0]
        path = mail[1][:-1]
        ret_indexes.append({"label": label,
                           "email_path": os.path.join(DATASET_PATH, path)})

    return ret_indexes
```

```
def parse_email(index):
    p = Parser()
    pmail = p.parse(index["email_path"])
    return pmail, index["label"]
```

```
indexes = parse_index("datasets/trec07p/full/index", 10)
indexes

[{'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.1'},
 {'label': 'ham', 'email_path': 'datasets/trec07p/data/inmail.2'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.3'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.4'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.5'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.6'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.7'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.8'},
 {'label': 'spam', 'email_path': 'datasets/trec07p/data/inmail.9'},
 {'label': 'ham', 'email_path': 'datasets/trec07p/data/inmail.10'}]
```

Preprocesamiento del DataSet

con las funciones presentadas anteriormente se permite la lectura de los correos electrónicos de manera programática y el procesamiento de los mismos para eliminar aquellos componentes que no resultan de utilidad para la detección de correos de SPAM. Sin embargo cada uno de los correos sigue estando representado por un diccionario de Python con una serie de palabras.

```
# Cargar el índice y las etiquetas en memoria
index = parse_index("datasets/trec07p/full/index", 1)
```



```
# leemos el primer correo
import os
open(index[0]["email_path"]).read()

'From: RickyAmes@aol.com Sun Apr 8 13:07:32 2007\nReturn-Path: <RickyAmes@aol.com>\nReceived: from 129.97.78.23 ([211.202.101.74])\n\tby speedy.uwaterloo.ca (8.12.8/8.12.5) with SMTP id l38H7G0I003017;\n\tSun, 8 Apr 2007 13:07:21 -0400\nReceived: from 0.144.152.6 by 211.202.101.74\n\t; Sun, 08 Apr 2007 19:04:48 +0100\nMessage-ID: <WYADCKPDFWWTXNFVUE@yahoo.com>\nFrom: "Tomas Jacobs" <RickyAmes@aol.com>\nReply-To: "Tomas J\n\tacobs" <RickyAmes@aol.com>\nTo: the00@speedy.uwaterloo.ca\nSubject: Generic Cialis, branded quality@\nDate: Sun, 08 Apr 2007 21:00:48 +0300\nX-Mailer: Microsoft Outlook Express 6.00.2600.0000\nMIME-Version: 1.0\nContent-Type: multipart/alternative;\n\tboundary="--8896484051606557286"\nX-Priority: 3\nX-MSMail-Priority: Normal\nStatus: RO\nContent-Length: 988\nLines: 24\n\n---8896484051606557286\nContent-Type: text/html;\nContent-Transfer-Encoding: 7Bit\n\n<html>\n<body bgcolor="#ffffff">\n<div style="border-color: #00FFFF; border-right-width: 0px; border-b\n\tottom-width: 0px; margin-bottom: 0px;" align="center">\n<table style="border: 1px; border-style: solid; border-color: #000000;" cellpadding="5\n\t" cellspacing="0" bgcolor="#CCFFAA">\n<tr>\n<td style="border: 0px; border-bottom: 1px; border-style: solid; border-color: #000000;">\n<center>\nDo you feel the pressure to perform and not rising to the occasion??<br>\n</center>\n</td>\n<tr>\n<td bgcolor="#FFFF33" style="border: 0px\n\t; border-bottom: 1px; border-style: solid; border-color: #000000;">\n<center>\n<b><a href='\"http://excortiationtuh.com/?lzmfnrdkleks\">Try <\n\tspan>V</span><span>i</span><span>a</span><span>g</span><span>r</span><span>a</span></td>\n</tr>\n</table>\n</div>\n</body>\n</html>\n\n\n\n---8896484051606557286--\n\n'
```

```
# parsear el primer correo
mail, label = parse_email(index[0])
print("El ecorreo es: ", label, "\n")
print(mail)

El ecorreo es: spam

{'subject': ['gener', 'ciali', 'brand', 'qualiti'], 'body': ['do', 'feel', 'pressur', 'perform', 'rise', 'occas', 'tri', 'viagra', 'anxieti', 'thing', 'past', 'back', 'old', 'self'], 'content_type': 'multipart/alternative'}
```

El algoritmo de Regresin Logistica no es capaz de ingerir texto como parte del DataSet. Por lo tanto, deben aplicarse una serie de funciones adicionales que transformen el texto de los correos electronicos parseados en una representacion numerica.

Aplicacion de CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer
#Preparacion del email en una cadena de texto.
prep_email = [" ".join(mail['subject']) + " ".join(mail['body'])]

vectorizer = CountVectorizer()
X = vectorizer.fit(prepare_email)
print("e-mail: ", prep_email, "\n")
print("Caracteristicas de entrada: ", vectorizer.get_feature_names_out())

e-mail: ['gener ciali brand qualitulo feel pressur perform rise occas tri viagra anxieti thing past back old self']

Caracteristicas de entrada: ['anxieti' 'back' 'brand' 'ciali' 'feel' 'gener' 'occas' 'old' 'past' 'perform' 'pressur' 'qualitulo' 'rise' 'self' 'thing' 'tri' 'viagra']
```

```
X = vectorizer.transform(prepare_email)
print("\nValues: \n", X.toarray())

Values:
[[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]]
```



Aplicacion de OneHotEncoding

```
from sklearn.preprocessing import OneHotEncoder

prep_email = [[w] for w in mail['subject'] + mail['body']]
enc = OneHotEncoder(handle_unknown = 'ignore')
X = enc.fit_transform(prepare_email)
print("Features:\n", enc.get_feature_names_out(), "\n")
print("\nValues:\n", X.toarray())
```

Features:

['x0_anxieti' 'x0_back' 'x0_brand' 'x0_ciali' 'x0_do' 'x0_feel' 'x0_gener'
'x0_occas' 'x0_old' 'x0_past' 'x0_perform' 'x0_pressur' 'x0_qualiti'
'x0_rise' 'x0_self' 'x0_thing' 'x0_tri' 'x0_viagra']

Values:

```
[[0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
```

Funciones auxiliares preprocesamiento del DataSet

```
def create_prep_dataset(index_path, n_elements):  
    X = []  
    y = []  
    indexes = parse_index(index_path, n_elements)  
    for i in range(n_elements):  
        print("\rParsing email: {0}".format(i+1), end = '')  
        mail, label = parse_email(indexes[i])  
        X.append(" ".join(['subject'])) + " ".join(mail['body']))  
        y.append(label)  
    return X, y
```

```
# Leer unicamente un subconjunto de 1000 correo electronicos  
X_train, y_train = create_prep_dataset("datasets/trec07p/full/index", 1000)  
X_train
```

Parsing email: 1000

```
['subjectdo feel pressur perform rise occas tri viagra anxieti thing past back old self',  
'subjecthi updat gulu i check mirror it seem littl typo debian readm file exampl http gulu usherbrook ca debian readm ftp ftp fr debian org  
debian readm test lenni access releas dist test the current test develop snapshot name etch packag test unstabl pass autom test propog releas  
etch replac lenni like readm html yan morin consult en logiciel libr yan morin savoirfairelinux com 514 994 1556 to unsubscrib email debian m  
error request list debian org subject unsubscrib troubl contact listmast list debian org',  
'subjectmega authenticv i a g r a discount pricec i a l i s discount pricedo miss it click http www moujsjkhchum com authent viagra mega aut  
henticv i a g r a discount pricec i a l i s discount pricedo miss it click',  
'subjectthey billi realli fun go night talk said felt insecur manhood i notic toilet quit small area worri websit i tell secret weapon extra  
3 inch trust girl love bigger one i 5 time mani chick sinc i use pill year ago the packag i use 6 month suppli one worth everi cent websit ht  
tp ctmay com ring weekend go drink let know secret later dude brad',  
'subjectsystem home it capabl link far i know i within part with respect affect technolog societi scienc ad agenc cashin g commerci photogra  
ph paint electron canvas still seem like silenc white black light it happen although far p erfect especu preclud vast explan i understand peo  
pl reli avant gard art world addit writer lawyer yet ach full potenti i imagin futur comput screen ho w would percept artifici imag busi apar  
t there computer sign shop filter unnecessari labor technolog comp ani monopoli servic as servic root tre nd western cultur sinc dark age pos  
sibl unimag in human think friendli not industri welcom new peopl thought c enter comput voic activ capabl mo pervas year go we email system  
organizatio n individu booth design architect draftspeopl condit i call hyper awar anyth everyth five i alway took prid e know reason i hire
```

Aplicar vectorizacion a los datos

```
vectorizer = CountVectorizer()  
X_train = vectorizer.fit_transform(X_train)  
  
print(X_train.toarray())  
print("\n Features", len(vectorizer.get_feature_names_out()))  
  
[[0 0 0 ... 0 0 0]  
 [0 0 0 ... 0 0 0]  
 [0 0 0 ... 0 0 0]  
 ...  
 [0 0 0 ... 0 0 0]  
 [0 0 0 ... 0 0 0]  
 [2 0 0 ... 0 0 0]]  
  
Features 17296
```




4. Prediccion

```
# Lectura de un DataSet de correos nuevos .
# Leer 150 de nuestro DataSet y quedarnos unicamente con los 500 ultimos correos electronicos, los cuales no se han utilizado
# para entrenar el algoritmo
X, y = create_prep_dataset("datasets/trec07p/full/index", 150)
X_test = X[100:]
y_test = y[100:]

Parsing email: 150
```

```
LogisticRegression()
```

Preprocesamiento de los correos electronicos cin el vectorizado creado anetriormente

```
X_test = vectorizer.transform(X_test)
```

```
y_pred = clf.predict(X_test)
y_pred
```

```
array(['spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam',
       'spam', 'spam', 'ham', 'spam', 'spam', 'spam', 'spam', 'spam',
       'ham', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'ham',
       'spam', 'spam', 'ham', 'spam', 'spam', 'ham', 'spam', 'spam',
       'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam',
       'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam',
       'spam', 'spam'], dtype='<U4')
```

```
print("Prediccion \n", y_pred)
print("\n Etiqueta Reales", y_test)
```

```
Prediccion
['spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam'
 'ham' 'spam' 'spam' 'spam' 'spam' 'spam' 'ham' 'spam' 'spam' 'spam'
 'spam' 'spam' 'spam' 'ham' 'spam' 'spam' 'ham' 'spam' 'spam' 'ham' 'spam'
 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam'
 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam' 'spam']

Etiqueta Reales ['spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'ham', 'spam', 'spam', 'spam', 'spam',
 'ham', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'ham', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam',
 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam', 'spam']
```

Evaluacion de resultados

```
from sklearn.metrics import accuracy_score
print("Accuracy: {:.3f}".format(accuracy_score(y_test, y_pred)))

Accuracy: 1.000
```

5.- Aumentando el DataSet

```
# Leer 20000 correos electronicos
X, y = create_prep_dataset("datasets/trec07p/full/index", 20000)

Parsing email: 20000
```

```
# Utilizamos 15 000 correos electronicos para entrenar el algoritmo y 5000 para realizar pruebas
X_train, y_train = X[:15000], y[:15000]
X_test, y_test = X[15000:], y[15000:]

vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(X_train)

clf = LogisticRegression()
clf.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?

LogisticRegression()

```
X_test = vectorizer.transform(X_test)
y_pred = clf.predict(X_test)

print("Accuracy: {:.3f}".format(accuracy_score(y_test, y_pred)))

Accuracy: 0.990
```

Conclusión:

La regresión logística es una herramienta sencilla y efectiva en machine learning para detectar correos ham y spam. Se utiliza para predecir la probabilidad de que un correo pertenezca a una de estas dos categorías. Funciona asignando pesos a diferentes características del correo como la frecuencia de ciertas palabras o la longitud del mensaje y calcula la probabilidad de que sea spam. Si esa probabilidad supera un umbral el mensaje se clasifica como spam y si no es ham. Aunque es un modelo simple en comparación con técnicas más complejas como los árboles de decisión o las redes neuronales la regresión logística es rápida interpretable y suele ofrecer resultados sólidos cuando se tienen buenas características de entrada.

NOMBRE DEL ALUMNO: Shania Kinnereth Diaz Moya

Grupo: 3501

Periodo: 2024-2

RÚBRICA DE EVALUACIÓN ISIC Tecnológico de Estudios Superiores de Jilotepec	Puntaje		 Comentarios
	DOCUMENTO: Prácticas	Máxima	Obtenida
Análisis de requerimientos Identifica los datos de relevancia y la problemática asociada.	20		
Diseño / modelado Representación gráfica o escrita de la solución	20		
Codificación / Configuración Traslado del modelado a un lenguaje de programación y/o configuración de la solución.	20		
Pruebas Comparar resultados obtenidos con los esperados	30		
Documentación de la práctica Entrega de manual de prácticas FO-ACA-11	10		
Tiempo de entrega No se califican entregables fuera de tiempo			



Calificación	100		
--------------	-----	--	--